

Healthcare Data Warehouse

Technical & Solution Design Documentation

End-to-End Analytics on Microsoft Fabric · Medallion Architecture



Connecting clinics, doctors, and patients through unified healthcare analytics

Prepared by **Mohamed Saber**

Data Engineer | Microsoft Fabric · Power BI · PySpark

July 2026

Table of Contents

1. Project Overview	3
1.1 Objectives	3
2. Data Overview	4
2.1 Source Datasets.....	4
2.2 Analytical Questions	7
3. Conceptual, Logical & Physical Model	10
3.1 Conceptual Model	10
3.2 Logical Model	10
3.3 Physical Model	11
4. Bronze Layer Overview	13
4.1 Read Source	13
4.2 Incremental Load to Bronze	13
5. Silver Layer Overview	14
5.1 Incremental Data Extraction	14
5.2 Validation.....	15
5.3 Merge to Source.....	15
6. Gold Layer Overview	16
6.1 Dimension Dataflow	16
6.2 Stored Procedures — SCD Type 2.....	17
6.3 Fact Dataflow	17
7. Power BI Overview	19
7.1 Key DAX Measures.....	19
7.2 Dashboard Pages.....	20
7.3 Summary.....	23

1. Project Overview

This document describes the end-to-end design and implementation of a Healthcare Data Warehouse built on Microsoft Fabric, covering the data model, the Medallion Architecture (Bronze, Silver, Gold), and the Power BI reporting layer.

1.1 Objectives

1. Design a scalable Healthcare Data Warehouse.
2. Implement Bronze, Silver, and Gold layers using the Medallion Architecture.
3. Implement incremental loading for all source datasets.
4. Apply Slowly Changing Dimension (SCD) Type 2 for the Patient and Doctor dimensions.
5. Build a Star Schema optimized for analytical workloads.
6. Develop a Power BI Semantic Model.
7. Create interactive executive dashboards.
8. Automate orchestration using Microsoft Fabric Pipelines.

2. Data Overview

Healthcare organizations generate large volumes of operational data, including patients, doctors, appointments, treatments, prescriptions, insurance information, and payments. This project implements a complete end-to-end healthcare analytics platform on Microsoft Fabric, following the Medallion Architecture. The solution ingests raw CSV and TXT files, transforms them using PySpark notebooks, builds a dimensional warehouse using Dataflow Gen2 and Microsoft Fabric Warehouse, and exposes a semantic model for Power BI reporting.

The platform supports executive reporting, operational analysis, revenue analysis, and clinical insights, while maintaining scalability, historical tracking, and automated refresh processes.

2.1 Source Datasets

The warehouse is built from eight source files, summarized below.

1. Appointments.csv

Contains all patient appointment records, including scheduling details, status, and the reason for each visit. This file is the transactional core of the model and is used to build the FactAppointment table.

Column	Description
AppointmentID	Unique appointment identifier
PatientID	Patient identifier
DoctorID	Doctor identifier
ClinicID	Clinic identifier
AppointmentDate	Date of the appointment
Status	Appointment status (Completed, Cancelled, etc.)
ReasonForVisit	Reason given by the patient for the visit

2. Patients.csv

Stores demographic and contact information for all patients. It provides descriptive attributes that support patient segmentation, insurance analysis, and revenue analysis.

Column	Description
PatientID	Unique patient identifier
FullName	Patient full name
Gender	Patient gender
DateOfBirth	Date of birth

Column	Description
Phone	Patient's phone number
Address	Patient's address
Emergency_contact	Phone number for emergency situations
BloodGroup	Blood type
RegistrationDate	Patient registration date

3. Doctors.csv

Contains information about healthcare providers. It is used to analyze doctor performance, specialty performance, appointment distribution, and revenue generation.

Column	Description
DoctorID	Unique doctor identifier
DoctorName	Doctor full name
Specialty	Medical specialty
Email	Doctor's email address
Phone	Contact number
Clinic_id	Clinic where the doctor works

4. Clinics.csv

Contains clinic master data and is used to analyze operational performance, appointment volume, and revenue across clinics.

Column	Description
ClinicID	Unique clinic identifier
ClinicName	Name of the clinic
City	Clinic location — city
State	Clinic location — state
Address	Full clinic address
Phone	Clinic contact number
Opening Hours	Hours during which the clinic is open
Clinic_type	Public or private

5. Treatments.csv

Stores information about the treatment types provided by healthcare clinics during appointments. It supports treatment analysis, revenue analysis, and operational reporting.

Column	Description
TreatmentID	Treatment identifier
Appointment_id	Related appointment identifier
Treatment_type	Type of treatment provided
Status	Completed, Pending, etc.
Treatment_cost	Cost of the treatment

6. Prescription.csv

Contains the medicines prescribed during appointments. It supports medicine utilization analysis and prescribing-pattern analysis.

Column	Description
PrescriptionID	Prescription identifier
Appointment_id	Related appointment identifier
MedicineName	Name of the prescribed medicine
Instruction	Prescribed dosage instructions

7. Payments.csv

Stores payment transactions associated with patient appointments. It provides the financial information used for revenue reporting and payment analysis.

Column	Description
PaymentID	Unique payment identifier
AppointmentID	Related appointment
PaymentDate	Date the payment was made
Payment amount	Amount paid
Payment method	Method used for payment
Payment_Status	Paid, Pending, etc.

8. Patient_Insurance.txt

Stores insurance information for patients, including insurance provider and coverage level. It enriches the patient dimension and supports insurance-based revenue analysis.

Column	Description
PatientID	Patient identifier
InsuranceProvider	Insurance company
Coverage level	Level of insurance coverage
Insurance policy Status	Insured or not insured

2.2 Analytical Questions

The following analytical questions define the reporting requirements for the FactAppointment star schema and drove the design of the dimension and measure layers.

Q1. Which doctor specialties generated the highest revenue in each clinic during the last quarter?

Business Value

- Identify the most profitable specialties across clinics.
- Support staffing and investment decisions.

Dimensions Used

DimDoctor • DimClinic • DimDate

Measure: **SUM(VisitAmount)**

Q2. Which treatment types generated the highest revenue across insurance providers during the last 12 months?

Business Value

- Evaluate treatment profitability by insurance provider.
- Support negotiations with insurance companies.

Dimensions Used

DimTreatment • DimPatient • DimDate

Measure: **SUM(VisitAmount)**

Q3. Which clinics performed the highest number of completed appointments by treatment type during the last six months?**Business Value**

- Measure operational efficiency.
- Compare clinic performance across treatment categories.

Dimensions Used

DimClinic • DimTreatment • DimAppointment

Measure: **COUNT(AppointmentID)**

Q4. Which doctors generated the highest revenue for patients with Premium insurance coverage during the previous year?**Business Value**

- Identify high-value doctors serving premium-insured patients.
- Support strategic planning.

Dimensions Used

DimDoctor • DimPatient • DimDate

Measure: **SUM(VisitAmount)**

Q5. Which medicines were prescribed most frequently by doctor specialty during the last six months?**Business Value**

- Analyze prescribing patterns by specialty.
- Improve medicine inventory planning.

Dimensions Used

DimMedicine • DimDoctor • DimDate

Measure: **COUNT(AppointmentID)**

Q6. Which medicines were prescribed most frequently in each clinic during the last quarter?**Business Value**

- Compare medication demand across clinics.
- Improve medicine distribution and procurement planning.

Dimensions Used

DimMedicine • DimClinic • DimDate

Measure: **COUNT(AppointmentID)**

Q7. Which appointment reasons had the highest cancellation rate during the previous year?

Business Value

- Identify the main causes of appointment cancellations.
- Improve scheduling efficiency and patient experience.

Dimensions Used

DimAppointment • DimDate

Measure: **COUNT(AppointmentID)**

Q8. What was the total revenue generated during the last quarter?

Business Value

- Measure overall financial performance.

Dimensions Used

DimDate

Measure: **SUM(VisitAmount)**

Q9. Which clinics handled the highest number of appointments during the last six months?

Business Value

- Measure clinic utilization.
- Support capacity planning.

Dimensions Used

DimClinic • DimDate

Measure: **COUNT(AppointmentID)**

Q10. What was the average revenue per patient during the last 12 months?

Business Value

- Measure patient value.
- Support pricing and financial planning.

Dimensions Used

DimPatient • DimDate

Measure: **AVG(VisitAmount)**

3. Conceptual, Logical & Physical Model

3.1 Conceptual Model

The conceptual model provides a high-level representation of the healthcare business process. It focuses on business concepts rather than technical implementation, showing how the central appointment event relates to the doctors, patients, clinics, treatments, medicines, and dates involved.

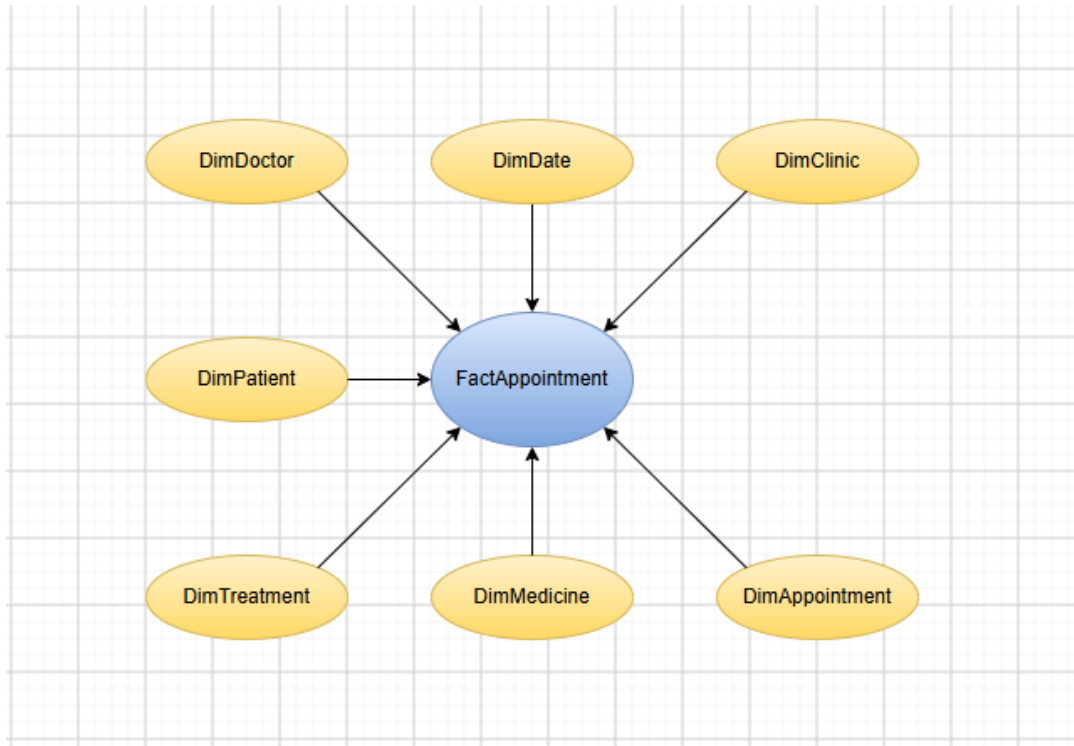


Figure 1 — Conceptual data model

3.2 Logical Model

The logical data model translates the conceptual business entities into a structured data model by defining entity attributes, primary keys, and relationships between dimensions and the fact table.

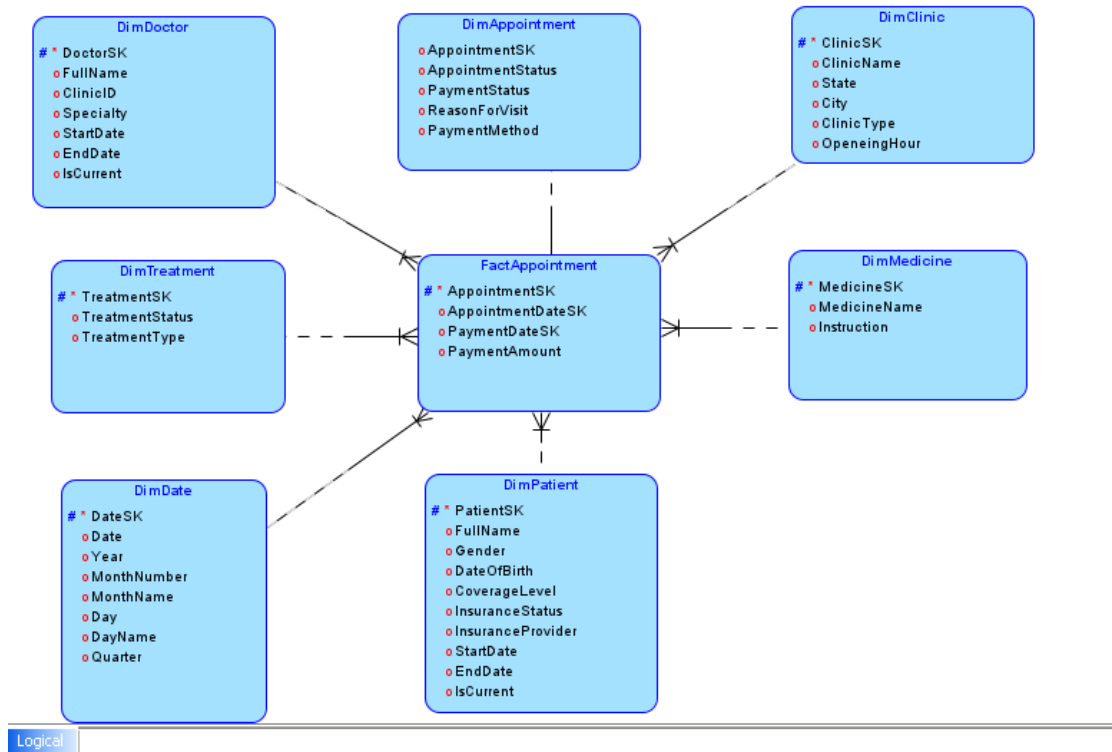


Figure 2 — Logical data model

3.3 Physical Model

The physical data model represents the final implementation of the analytical warehouse inside the Microsoft Fabric Warehouse. The warehouse follows a Star Schema design that separates descriptive information into dimension tables while storing transactional information inside a centralized fact table. This design significantly improves reporting performance and simplifies analytical queries.

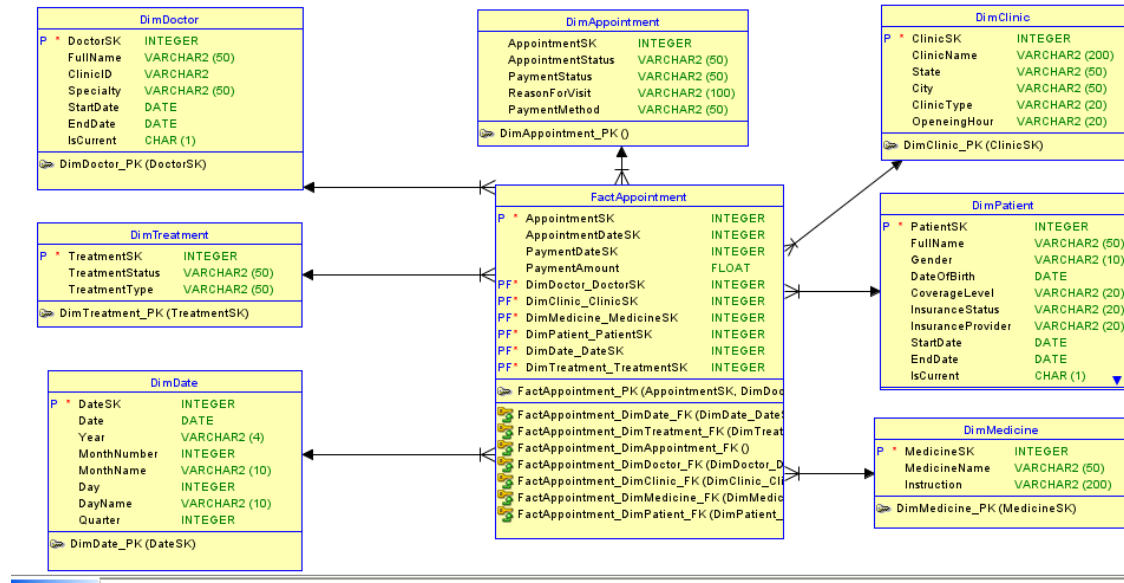


Figure 3 — Physical star-schema model

Description of Tables

FactAppointment (Fact Table): Stores appointment transactions, payment amounts, and references to patients, doctors, clinics, treatments, medicines, and dates.

DimPatient (Patient Dimension): Contains patient demographic and insurance information used for patient and insurance analysis. Tracked as SCD Type 2.

DimDoctor (Doctor Dimension): Stores doctor information including specialty and clinic, supporting doctor performance analysis. Tracked as SCD Type 2.

DimClinic (Clinic Dimension): Contains clinic details such as name, location, and clinic type for operational analysis.

DimTreatment (Treatment Dimension): Stores treatment information used for treatment and service performance analysis.

DimMedicine (Medicine Dimension): Contains medicine details used for prescription and medicine utilization analysis.

DimAppointment (Appointment Dimension): Stores appointment attributes including status, payment status, payment method, and reason for visit.

DimDate (Date Dimension): Contains calendar attributes used for time-based analysis. Supports both Appointment Date and Payment Date as a role-playing dimension.

4. Bronze Layer Overview

The Bronze Layer is the raw data ingestion layer of the Healthcare Data Warehouse. It is responsible for importing the source CSV and TXT files into the Microsoft Fabric Lakehouse without applying any business transformations. Data is stored as Delta tables in its original format to preserve the raw source for auditing and reprocessing purposes. Incremental loading is implemented using PySpark notebooks and Delta Lake MERGE operations, while metadata columns such as `load_datetime` and `source_file` are added to support data lineage and incremental processing.

4.1 Read Source

The pipeline reads each source file from the landing area into the Lakehouse.

✓ Import Source Function

```
> | v
1  from pyspark.sql.functions import current_timestamp, input_file_name
2
3  def read_source(table_name):
4      return (
5          spark.read
6              .option("header", True)
7              .option("inferSchema", True)
8              .csv(f"Files/{table_name}.csv")
9              .withColumn("load_datetime", current_timestamp())
10             .withColumn("source_file", input_file_name())
11         )
[1] ✓ - Session ready in 10 sec 952 ms. Command executed in 346 ms by mohamed saber on 8/2/2026, 9:19:53 PM
...
```

Figure 4 — Read source files

4.2 Incremental Load to Bronze

Data cleansing and transformation steps are applied during the incremental load to Bronze.

Incremental Load To Bronze



```
1 from delta.tables import DeltaTable
2 def incremental_load(table_name, primary_key):
3
4     src_df = read_source(table_name)
5
6     tgt = DeltaTable.forName(spark, f"bronze.{table_name}")
7
8     (
9         tgt.alias("tgt")
10        .merge(
11            src_df.alias("src"),
12            f"src.{primary_key} = tgt.{primary_key}"
13        )
14        .whenNotMatchedInsertAll()
15        .execute()
16    )
17
18    print(f"{table_name} Incremental Load Completed.")
```

[9] ✓ - Command executed in 276 ms by mohamed saber on 8/2/2026, 9:49:34 PM

...

```
1 for table_name, primary_key in tables.items():
2     incremental_load(table_name, primary_key)
```

[10] ✓ - Command executed in 41 sec 200 ms by mohamed saber on 8/2/2026, 9:50:19 PM

Figure 5 — Incremental load to Bronze

5. Silver Layer Overview

The Silver Layer is the data cleansing and transformation layer of the Healthcare Data Warehouse. It receives raw data from the Bronze Layer and applies data quality rules to produce clean, standardized, and validated datasets. Using PySpark notebooks, the layer removes duplicates, handles missing values, standardizes data formats, validates records, corrects data types, and merges incremental data into Silver Delta tables. The cleansed data serves as the trusted source for building the dimensional model in the Gold layer.

5.1 Incremental Data Extraction

▼ Incremental Data Extraction Function

```

1  import pyspark.sql.functions as f
2
3  def get_incremental_data(table_name):
4      """
5      Reads Bronze and Silver tables, retrieves the latest load timestamp
6      from the silver table, and returns only the new records from Bronze.
7      """
8
9      # Read Bronze table
10     bronze_df = spark.read.table(f"Bronze.{table_name}")
11
12     # Read Silver table
13     silver_df = spark.read.table(f"Silver.{table_name}")
14
15     # Get the latest load timestamp from Silver
16     last_load = (
17         silver_df
18         .agg(f.max("load_datetime").alias("last_load"))
19         .collect()[0]["last_load"]
20     )
21
22     # First load -> return all Bronze records
23     if last_load is None:
24         return bronze_df
25
26     # Incremental load -> return only new records
27     return bronze_df.filter(
28         f.col("load_datetime") > last_load
29     )

```

[1] ✓ - Session ready in 10 sec 912 ms. Command executed in 320 ms by mohamed saber on 8/2/2026, 11:09:30 PM

...

Figure 6 — Incremental data extraction

5.2 Validation

Validation Function

```

1 def validate_dataframe(df, table_name, primary_key):
2
3     print("-" * 60)
4     print(f"({table_name}) Validation Report")
5     print("-" * 60)
6
7     # Row Count
8     row_count = df.count()
9     print(f"Total Rows: {row_count}")
10
11    # Duplicate Primary Keys
12    duplicate_count = {
13        df.groupby(primary_key)
14        .count()
15        .filter(f.col("count") > 1)
16        .count()
17    }
18
19    print(f"Duplicate {primary_key}: {duplicate_count}")
20
21    # Null Values
22    print("\nNull Values Per Column:")
23
24    display(
25        df.select(
26            {
27                f.count(
28                    f.when(f.col(col).isNull(), col)
29                ).alias(col)
30                for col in df.columns
31            }
32        )
33    )
34
35    # Sample Data
36    print("\nSample Data:")
37

```

Figure 7 — Data validation

5.3 Merge to Source

Merge Data into Silver Function

```

1 from delta.tables import DeltaTable
2
3 def merge_to_silver(table_name, clean_df, primary_key):
4     """
5     Merge cleaned data into the Silver table.
6
7     Parameters
8     -----
9     table_name : str
10         Silver table name.
11     clean_df : DataFrame
12         Cleaned dataframe to merge.
13     primary_key : str
14         Primary key column used for matching.
15     """
16
17     silver_table = DeltaTable.forName(
18         spark,
19         f"Silver.{table_name}"
20     )
21
22     merge_condition = {
23         f"tgt.{primary_key} = src.{primary_key}"
24     }
25
26     update_columns = {
27         col: f"src.{col}"
28         for col in clean_df.columns
29     }
30
31     (
32         silver_table.alias("tgt")
33         .merge(
34             clean_df.alias("src"),
35             merge_condition
36         )
37         .whenMatchedUpdate(set=update_columns)
38         .whenNotMatchedInsert(values=update_columns)
39         .execute()

```

Figure 8 — Merge to source

6. Gold Layer Overview

The Gold Layer is the business-ready layer of the Healthcare Data Warehouse. It transforms the cleansed data from the Silver Layer into a dimensional model optimized for analytics and reporting. Using Dataflow Gen2, dimension and fact tables are created inside the Microsoft Fabric Warehouse following a Star Schema design. Surrogate keys are generated for all dimensions, while SCD Type 2 is implemented for the Patient and Doctor dimensions using stored procedures to preserve historical changes. The Gold layer serves as the trusted source for the semantic model and Power BI dashboards.

6.1 Dimension Dataflow

The dimension dataflow is responsible for transforming the cleansed data from the Silver Layer into business-ready dimension tables within the Microsoft Fabric Warehouse. Each dimension is created through a dedicated Dataflow Gen2 that reads data from the Silver Lakehouse, applies the required transformations, removes duplicate records, and loads the results into staging tables. Surrogate keys are automatically generated using identity columns in the warehouse. For dimensions that require historical tracking, such as DimPatient and DimDoctor, the dataflow loads the data into temporary staging tables, where stored procedures apply SCD Type 2 logic before updating the final dimension tables.

Operations Performed

- Read data from the Silver Lakehouse.
- Select only the required business columns.
- Remove duplicate records.
- Load data into staging tables in the warehouse.
- Generate surrogate keys using identity columns.
- Apply SCD Type 2 for the Patient and Doctor dimensions using stored procedures.
- Load static dimensions directly into the final dimension tables.

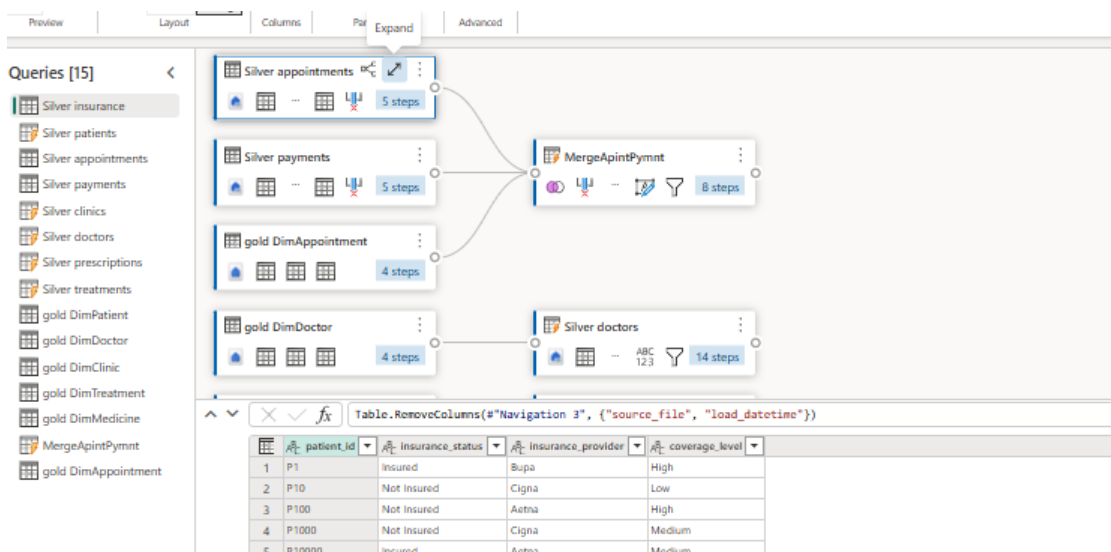


Figure 9 — Dimension dataflow

6.2 Stored Procedures — SCD Type 2

The stored procedures are responsible for implementing SCD Type 2 for the Patient and Doctor dimensions. After the dimension dataflow loads the latest records into staging tables, the stored procedures compare the incoming data with the existing dimension records. If changes are detected, the current record is expired and a new version is inserted while preserving the historical data. This approach ensures a complete change history and maintains data integrity within the dimensional model.

Operations Performed

- Read data from the staging tables.
- Compare incoming records with the current dimension records.
- Detect changes in tracked attributes.
- Expire the existing record by updating its end date and current flag.
- Insert a new version of the changed record.
- Ignore unchanged records to avoid unnecessary updates.

```

14  -- Doctors Stored Procedure
15  CREATE OR ALTER PROCEDURE gold.usp_Load_DimDoctor
16  AS
17  BEGIN
18
19      -----
20      -- Close Current Records
21      -----
22      UPDATE D
23      SET
24          D.EndDate = CURRENT_TIMESTAMP,
25          D.IsCurrent = 0
26      FROM gold.DimDoctor D
27      INNER JOIN gold.DimDoctor_Changes C
28          ON D.DoctorID = C.DoctorID
29      WHERE
30          C.RowStatus = 'Changed'
31          AND D.IsCurrent = 1;
32
33      -----
34      -- Insert New & Changed Records
35      -----
36      INSERT INTO gold.DimDoctor
37      (
38          DoctorID,
39          FullName,
40          Specialty,
41          ClinicID,
42          StartDate,
43          EndDate,
44          IsCurrent
45      )
46      SELECT

```

Figure 10 — SCD Type 2 stored procedure logic

6.3 Fact Dataflow

The fact dataflow is responsible for building the FactAppointment table in the Microsoft Fabric Warehouse. It reads the cleansed appointment data from the Silver Lakehouse, retrieves the corresponding surrogate keys from all dimension tables, and creates a business-ready fact table following the Star Schema design. The dataflow joins the transactional data with the dimensions, replaces business keys with surrogate keys, and loads the final fact records into the warehouse for analytical reporting.

Operations Performed

- Read appointment transactions from the Silver Lakehouse.
- Join appointment data with all dimension tables.
- Replace business keys with surrogate keys.
- Map Appointment Date and Payment Date to the DimDate role-playing dimension.
- Select the required measures and foreign keys.
- Load the final records into the FactAppointment table in the warehouse.

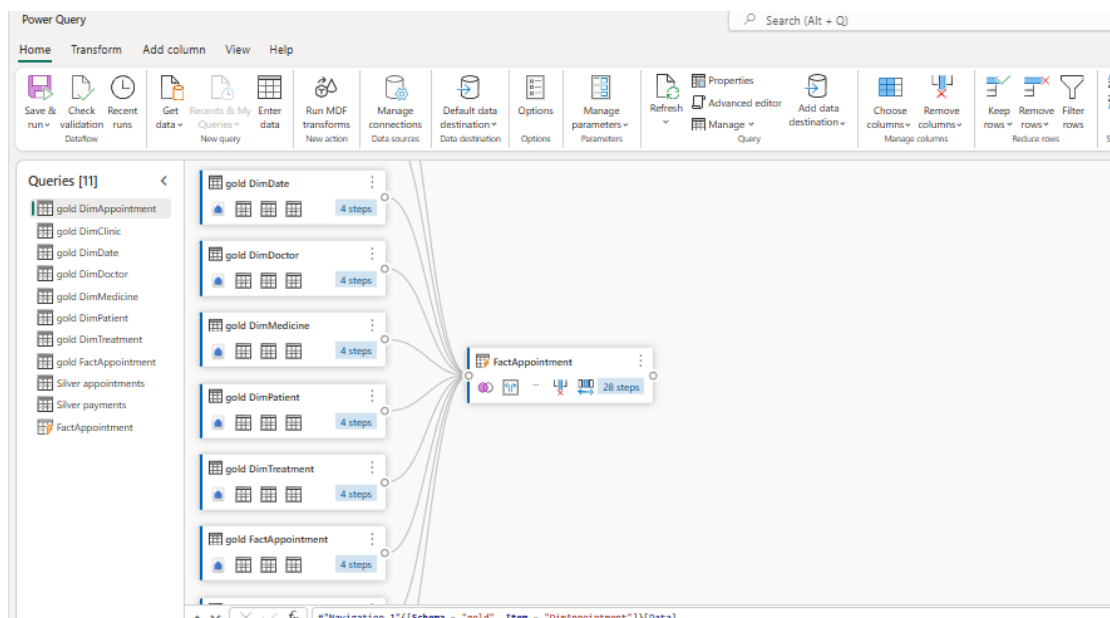


Figure 11 — Fact dataflow

7. Power BI Overview

The Power BI report connects directly to the Microsoft Fabric Semantic Model to provide interactive dashboards for appointment analysis, revenue tracking, clinical operations, doctor performance, and patient insights.

7.1 Key DAX Measures

Measure	DAX Expression
Appointments Measures	
Total Appointments	<code>DISTINCTCOUNT(FactAppointment[AppointmentSK])</code>
Completed Appointments	<code>CALCULATE([TotalAppointments], DimAppointment[AppointmentStatus] = "Completed")</code>
Cancelled Appointments	<code>CALCULATE([TotalAppointments], DimAppointment[AppointmentStatus] = "Cancelled")</code>
Cancellation Rate	<code>DIVIDE([CancelledAppointments], [TotalAppointments])</code>
Revenue Measures	
Total Revenue	<code>SUM(FactAppointment[PaymentAmount])</code>
Avg Revenue	<code>AVERAGE(FactAppointment[PaymentAmount])</code>
Avg Revenue per Appointment	<code>DIVIDE([Total Revenue], [Total Appointments])</code>
Avg Revenue per Clinic	<code>DIVIDE([Total Revenue], [Total Clinics])</code>
Avg Revenue per Doctor	<code>DIVIDE([Total Revenue], [Total Doctors])</code>
Avg Revenue per Patient	<code>DIVIDE([Total Revenue], [Total Patients])</code>
Patient Measures	
Male Patients	<code>CALCULATE([Total Patients], DimPatient[Gender] = "Male")</code>
Female Patients	<code>CALCULATE([Total Patients], DimPatient[Gender] = "Female")</code>
Premium Patients	<code>CALCULATE([Total Patients], DimPatient[CoverageLevel] = "High")</code>
Payment Measures	
Paid Appointments	<code>CALCULATE([TotalAppointments], DimAppointment[PaymentStatus] = "Paid")</code>
Paid Revenue	<code>CALCULATE([TotalRevenue], DimAppointment[PaymentStatus] = "Paid")</code>
Pending Revenue	<code>CALCULATE([TotalRevenue], DimAppointment[PaymentStatus] = "Pending")</code>

Measure	DAX Expression
Other Measures	
Total Clinics	<code>DISTINCTCOUNT(FactAppointment[ClinicSK])</code>
Total Doctors	<code>DISTINCTCOUNT(FactAppointment[DoctorSK])</code>
Total Medicines	<code>DISTINCTCOUNT(FactAppointment[MedicineSK])</code>
Total Treatments	<code>DISTINCTCOUNT(FactAppointment[TreatmentSK])</code>

7.2 Dashboard Pages

1 — Home / Landing Page

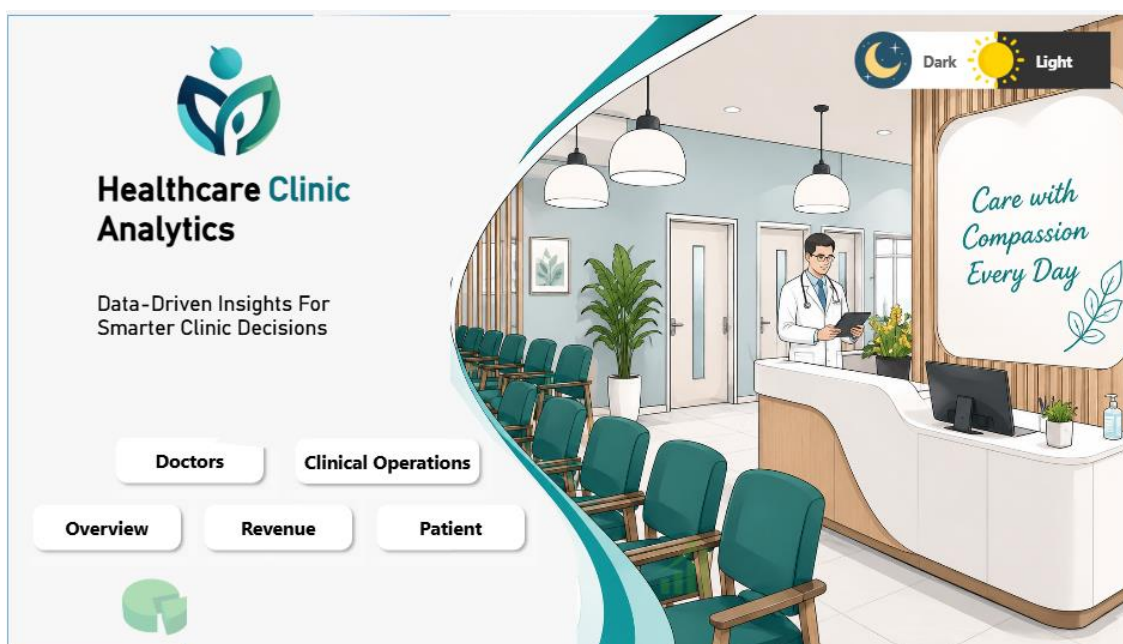


Figure 12 — Home landing page

2 — Overview

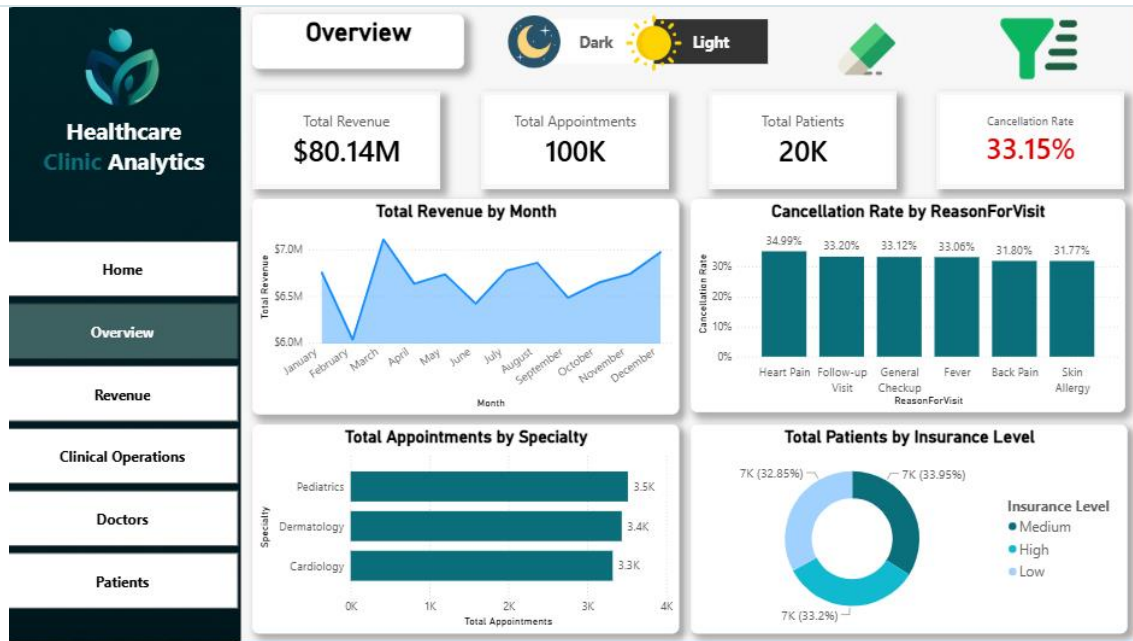


Figure 13 — Overview dashboard

3 — Revenue

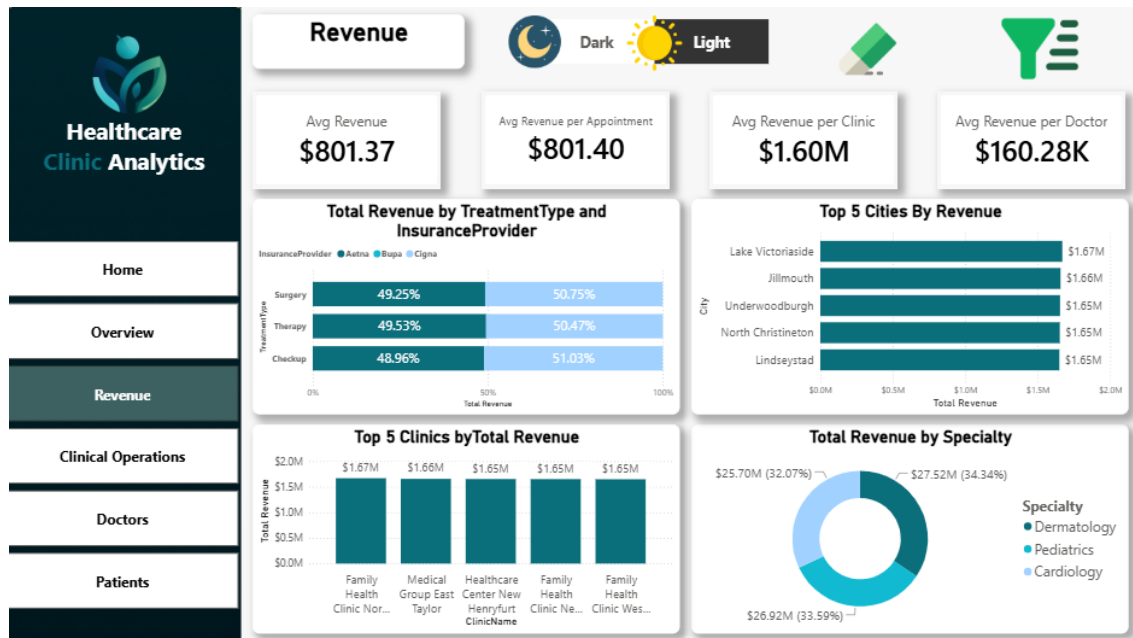


Figure 14 — Revenue dashboard

4 — Clinical Operations

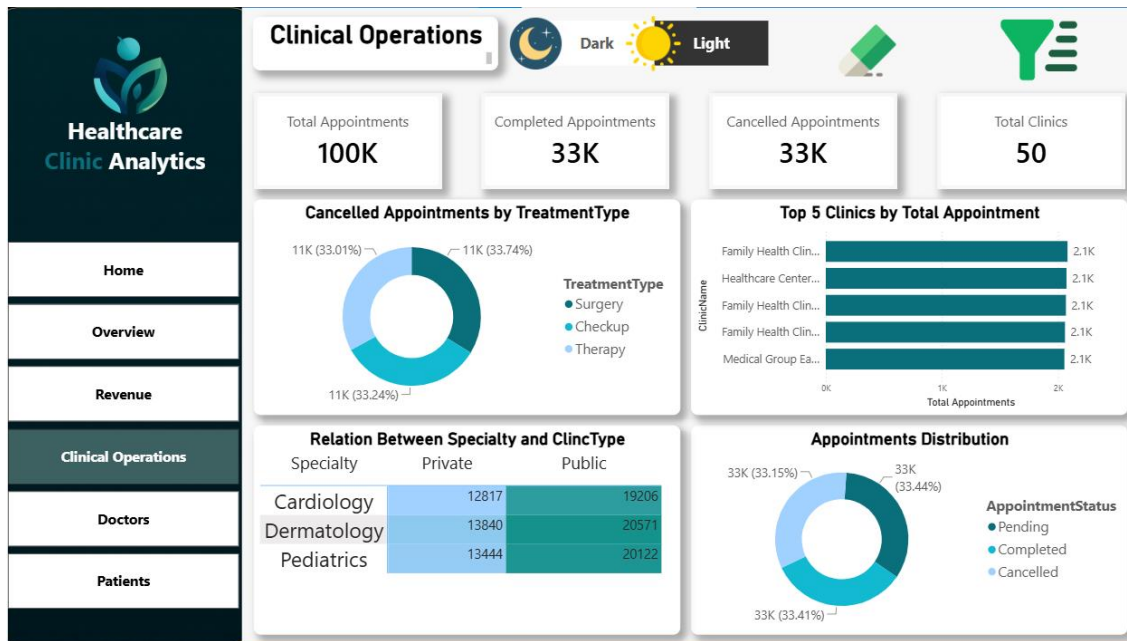


Figure 15 — Clinical operations dashboard

5 — Doctors



Figure 16 — Doctors dashboard

6 — Patients

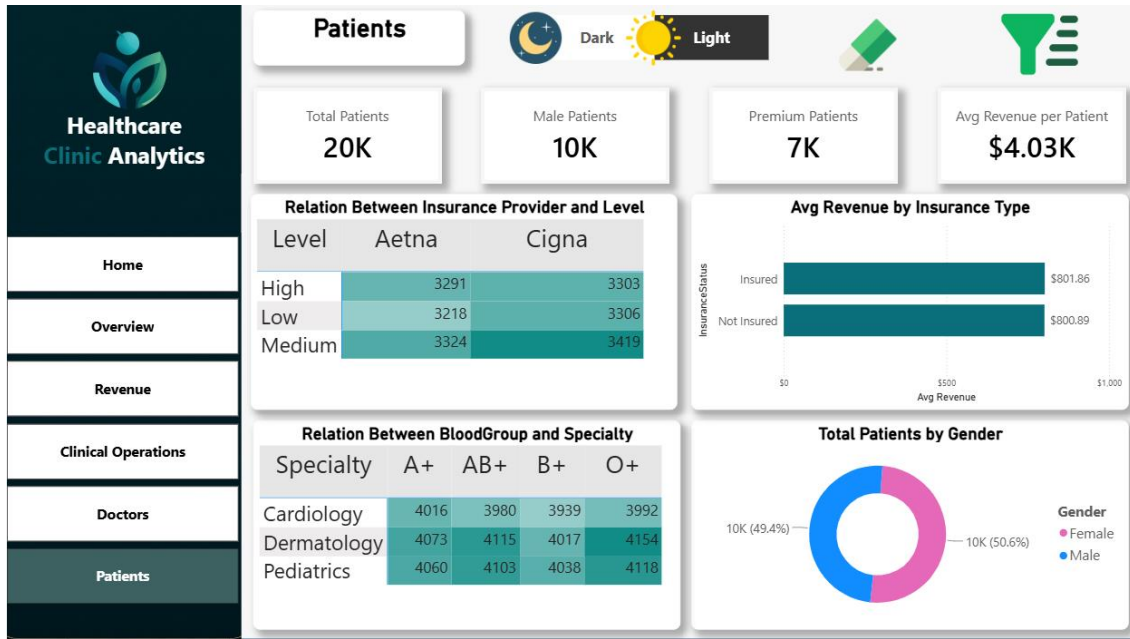


Figure 17 — Patients dashboard

7.3 Summary

The Healthcare Data Warehouse delivers a complete, governed analytics platform built on Microsoft Fabric, moving raw operational files through a Medallion Architecture into a trusted, business-ready star schema. Incremental loading keeps the warehouse current, SCD Type 2 preserves the full history of patients and doctors, and the Power BI semantic model turns that data into decision-ready dashboards for clinic operations, revenue, doctor performance, and patient insights.

Together, these components give stakeholders a single, reliable source of truth — reducing manual reporting effort, improving data quality, and enabling faster, evidence-based decisions across every clinic in the network.

HEALTHCARE ANALYTICS PLATFORM

Thank You

For reviewing the Healthcare Clinics documentation

Built on the Medallion Architecture — Bronze · Silver · Gold — with Microsoft Fabric and Power BI

Mohamed Saber

Data Engineer | Microsoft Fabric · Power BI · PySpark

July 2026

Healthcare Clinics · Technical & Solution Design Documentation